ChatGPT

# Designing a Trustworthy Money-Movement Application for a Ten-Million-User Future

## Executive summary

Because the research topic was explicitly left unspecified, three plausible topics span the requested domains:

| Domain | Plausible research topic | Why it is a strong candidate |
|---|---|---|
| **Technology** | **How to design a correct, resilient, scalable money-movement application** | The uploaded PSTU hackathon brief asks for exactly this problem: transfers and payment requests in a closed-money ecosystem, with correctness under concurrency, unreliable networks, and a possible future population above 10 million users. filecite turn0file0 |
| **Health** | **Clinical effectiveness, safety, and health-system implications of GLP-1-based obesity treatments** | This is a fast-moving field with meaningful clinical, economic, access, and long-term safety questions suitable for evidence synthesis. |
| **Policy** | **How governments should regulate instant-payment and digital-wallet ecosystems while preserving competition and financial inclusion** | This creates a useful policy comparison across consumer protection, fraud liability, interoperability, privacy, and systemic resilience. |

This report proceeds with the **technology topic**, because it is both the first proposal and directly applicable to the supplied challenge.

The central conclusion is that the strongest hackathon solution is **not a large microservice architecture**. It is a **modular monolith backed by one transactional relational database**, with an append-only double-entry ledger, explicit concurrency control, idempotent transfer submission, server-enforced transaction authorization, a state machine for money requests, and an outbox for reliable notifications. This architecture maximizes demonstrable correctness within the hackathon's deliberately constrained scope while retaining a credible path toward much larger scale. PostgreSQL's current implementation offers full serializable isolation with retry semantics when needed, while its documentation explicitly discusses account-transfer concurrency as a transactional use case. [1]

The most important engineering finding is that **a money transfer is not CRUD**. A correct transfer must preserve invariants even when two users spend simultaneously, a request is submitted twice, the

application times out after the database has committed, or a notification system fails. Dedicated financial-ledger systems such as TigerBeetle similarly model transfers as immutable records, maintain equality between debits and credits, and use identifiers for reliable, idempotent submission, illustrating how central these ideas are to financial transaction processing. [2]

For the challenge itself, the confidence that **transactional PostgreSQL plus a double-entry ledger is the best implementation choice is medium-high**: it is an architectural judgment rather than a universal theorem. Confidence is **high** that idempotency, atomicity, authorization, and concurrency handling are essential correctness properties; these are strongly supported by database documentation, OWASP guidance, NIST guidance, financial-ledger implementations, and recent peer-reviewed systems research. [3]

## Topic selection and background

The PSTU challenge defines a closed ecosystem using simulated funds rather than real banks or payment gateways. Every registered user has an account and balance, users can send money to one another or request money, the application may eventually serve more than 10 million users, and new users may begin with BDT 100,000. The brief specifically warns teams to think beyond a simplistic CRUD application because rapid user actions, concurrency, unpredictable networks, unexpected input, and growth can produce failures that ordinary database forms do not expose. It also explicitly favors a small, thoughtful working product over a broad but incomplete banking platform. 🗄️filecite🗄️turn0file0🗄️

That framing fundamentally changes the design objective. The goal is not merely:

> read A's balance → subtract money → read B's balance → add money.

The goal is to guarantee an invariant such as:

$$\text{Total debits posted} = \text{Total credits posted}$$

while also ensuring that a single logical transfer cannot accidentally execute twice and that no intermediate state becomes visible in which one account has been debited without the corresponding credit.

Double-entry financial modeling is particularly useful here. TigerBeetle's official financial-database documentation describes each transfer as an immutable debit from one account and credit to another and highlights equality of total debits and credits as a core accounting invariant. Its transfer records cannot be edited or deleted after creation; errors are corrected by new transfers rather than rewriting history. [4]

The same underlying principle can be implemented in ordinary PostgreSQL. PostgreSQL 18 supports atomic ACID transactions and serializable isolation; at `SERIALIZABLE`, successfully committed concurrent transactions are constrained to an outcome consistent with some serial execution, although applications must be prepared to retry serialization failures. [1] Recent peer-reviewed systems work reinforces that strong transactions need not inherently prevent large-scale operation: Amazon's DynamoDB transaction paper reports atomic, serializable transactions in a production distributed key-value service, while ScaleDB demonstrated scalable serializable transaction processing in an in-memory research system. These results do **not** benchmark the proposed hackathon application directly, but they weaken the simplistic assumption that correctness must be sacrificed to obtain scale. [5]

## Key questions and methodology

The research evaluated six questions: how balances should be represented; how simultaneous transfers should be serialized; how duplicate or retried requests should behave; how payment requests differ from transfers; where authentication and authorization belong; and whether the stated ten-million-user future requires distributed infrastructure from the beginning.

Sources were prioritized in the following order: the supplied challenge specification; current primary documentation from PostgreSQL, NIST, OWASP, AWS, IETF and financial-ledger implementations; then peer-reviewed systems papers from USENIX/CIDR published within roughly the last five years. PostgreSQL 18 documentation was checked rather than relying on historical behavior because transaction semantics and supported versions can change. OWASP's current ASVS page identifies version 5.0.0 as its latest stable Application Security Verification Standard, and NIST SP 800-63B-4 was published in August 2025. [6]

The quantitative analysis is deliberately **scenario-based rather than predictive** because the challenge specifies the number of future users but gives no transaction-frequency distribution. At 10 million users, three illustrative activity cases were modeled: 0.5, 2 and 10 transfers per user per day, together with a deliberately conservative 20× peak-to-average factor. Those numbers are capacity-planning assumptions, not empirical forecasts. The challenge's BDT 100,000 starting allocation implies BDT 1 trillion of simulated account balances if all 10 million eventual users received that amount, illustrating why even fake money benefits from rigorous reconciliation. 【filecite】turn0file0【】

## Findings and data

### Correctness should come from a ledger, not from the UI balance

A mutable `balance` column alone gives the application no independently replayable explanation of how the current value arose. The recommended model therefore makes **immutable ledger entries the accounting history** and treats the displayed balance as a transactional projection or cached aggregate.

For a transfer of 2,500 units from A to B, one logical transfer creates two corresponding ledger effects:

| Transfer | Account | Effect | Amount |
|---|---|---|---|
| T123 | A | Debit | 2,500 |
| T123 | B | Credit | 2,500 |

The application should enforce `amount > 0`, sender ≠ receiver, sufficient available funds, one currency/ledger per transfer, and equal debit/credit totals. Correcting a completed transfer should produce a reversal rather than rewriting historical entries. This recommendation follows the same accounting invariants used by purpose-built financial transaction databases, where transfers are immutable and corrections are modeled as new transfers. [4]

**Confidence: high.**

### Concurrency control is the difference between a wallet and CRUD

Consider an account with 1,000 available units. Two 800-unit transfers arrive concurrently. If each request independently reads 1,000 before either writes, naive application logic can approve both.

A simple relational implementation should open one database transaction, lock the relevant account rows, re-check the sender's available balance while those locks are held, write the transfer and both ledger entries, update any cached balances, and commit atomically. I recommend acquiring account locks in a deterministic account-ID order; that specific ordering is an engineering inference intended to reduce deadlock cycles.

PostgreSQL explains that `READ COMMITTED` can work correctly for predetermined account-row updates and even uses a two-account money transfer as an example, while more complex cross-row business rules can require stronger isolation. `SERIALIZABLE` provides the strictest PostgreSQL isolation and rejects executions that would create serialization anomalies, with applications expected to retry SQLSTATE `40001` failures. [1]

For a hackathon, two defensible options therefore exist:

| Strategy | Correctness potential | Complexity | Hackathon fit |
|---|---|---|---|
| Plain reads + independent updates | Poor under races | Low | **Reject** |
| Transaction + deterministic row locks | High when invariants are local | Moderate | **Excellent** |
| PostgreSQL `SERIALIZABLE` + retry loop | Very high/general | Moderate | Excellent |
| Distributed transaction layer from day one | Potentially high | Very high | Poor |

**Confidence: high** that explicit transactional concurrency control is mandatory; **medium-high** on row-locking versus serializable isolation because either can be correct when implemented rigorously. [1]

### Idempotency is mandatory because network failure creates ambiguity

A particularly dangerous failure occurs when the database successfully commits a transfer but the HTTP response never reaches the phone. The user taps **Send** again. Without an idempotency mechanism, a valid retry becomes a second transfer.

The application should require a client-generated `operation_id`/idempotency key and place a unique constraint on it. A retry carrying the same key and same logical payload returns the original transfer result rather than moving money again. A reused key with a different payload should be rejected.

This pattern is directly documented by financial transaction systems: TigerBeetle recommends generating and persisting a transfer ID on the client before submission and reusing it on retries, while Stripe

recommends idempotency keys to prevent duplicate payment intents. An IETF working-group draft formalized the same concept for HTTP POST/PATCH requests, although the latest draft expired in April 2026 and therefore should be treated as prior work rather than an active standard. [7]

**Confidence: very high.**

## Transaction authorization must be server-side

A client must never be allowed to say, in effect, `"senderId": 42` and have the server assume that the authenticated user owns account 42. The server derives the payer from the authenticated principal, verifies the destination and amount, enforces permitted state transitions, and performs a final authorization check immediately before executing the financial transaction.

OWASP's transaction-authorization guidance explicitly requires server-side enforcement, server-generated verification data, controlled transaction state transitions, protection against modification between approval and execution, and verification that every transaction was properly authorized. It also recommends showing the user the significant transaction data—particularly target account and amount—during confirmation. [8]

For stronger authentication, NIST's 2025 Digital Identity Guidelines make phishing-resistant authentication available at AAL2 and mandatory at AAL3; NIST identifies cryptographic authentication such as WebAuthn-style approaches as phishing-resistant, while manually entered OTPs are not considered phishing-resistant. NIST is a security benchmark here, **not a statement of Bangladesh-specific legal requirements**. [9]

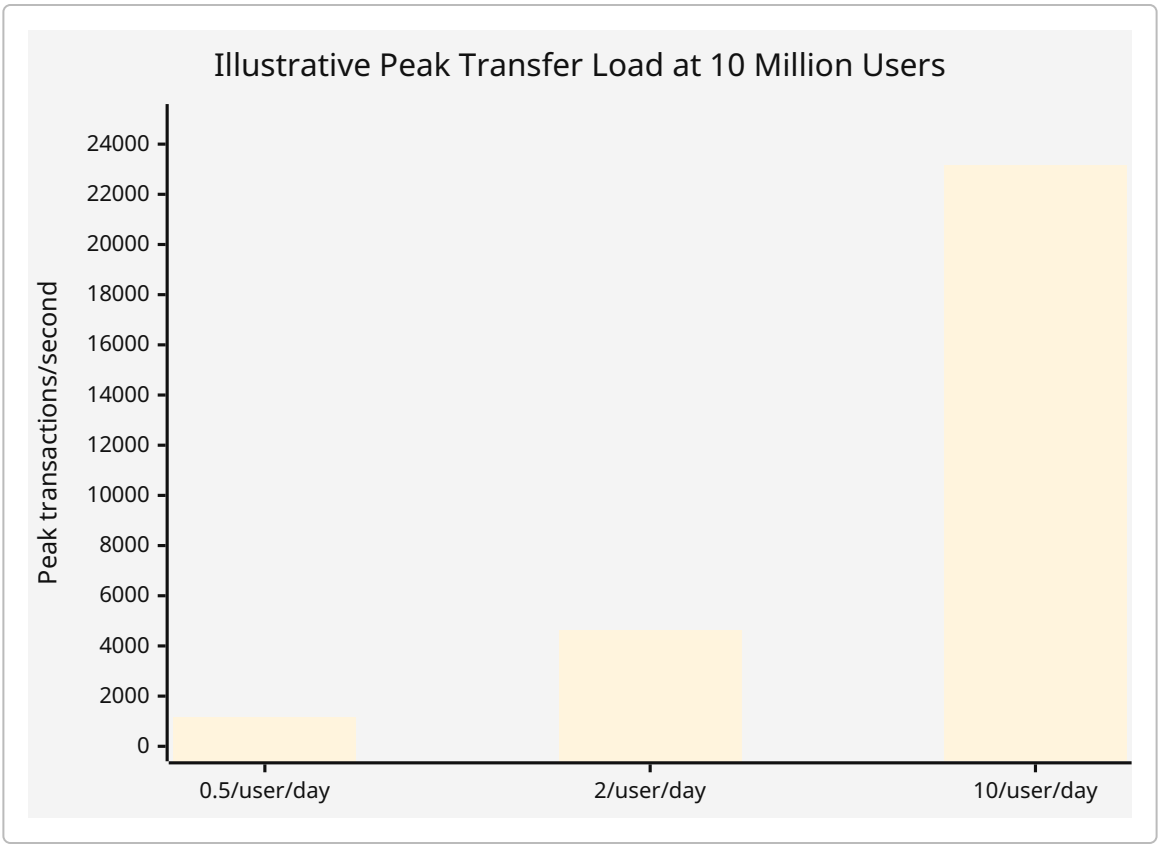
**Confidence: high.**

## Ten million users does not automatically mean microservices

The user-count target is large, but database load depends primarily on active transaction volume, skew, query patterns, data retention, and latency objectives—not registered-user count alone. Under the illustrative assumptions below, the range is broad.

| Activity assumption | Transfers/day | Average TPS | Illustrative 20× peak TPS |
|---|---|---|---|
| 0.5 transfer/user/day | 5.0 million | ~58 | ~1,157 |
| 2 transfers/user/day | 20.0 million | ~231 | ~4,630 |
| 10 transfers/user/day | 100.0 million | ~1,157 | ~23,148 |

These figures are calculated from the challenge's ten-million-user scenario and are not workload measurements. filecite turn0file0



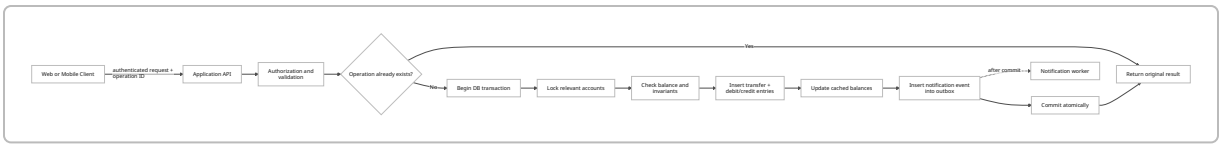

At the middle assumption, 20 million transfers per day would produce 40 million debit/credit ledger entries per day and roughly 14.6 billion entries per year. At an illustrative 200 bytes per ledger row before indexes, replication, backups and database overhead, that is about 2.9 TB of raw ledger data annually. These values establish that a future implementation may eventually need partitioning, archival policies and perhaps a specialized ledger store, but they do **not** justify burdening a hackathon prototype with distributed infrastructure prematurely.

Recent research demonstrates multiple routes to scalable transactional processing rather than one mandated architecture. DynamoDB supports ACID distributed transactions using timestamp ordering, while ScaleDB reports substantially higher benchmark throughput than comparison systems while retaining serializability; CIDR work continues to investigate both geo-distributed strictly serializable transaction systems and cloud OLTP architectures. [10]

**Confidence: medium-high** that the prototype should remain single-database; **low** confidence in any exact future throughput requirement because transaction-frequency data are absent.

## Architecture and implementation implications

For the competition, the recommended architecture is deliberately compact:




The core tables can remain small: `users`, `accounts`, `transfers`, `ledger_entries`, `money_requests`, and `outbox_events`. The transfer table should contain a globally unique transfer ID, idempotency key, sender and receiver accounts, amount in integer minor units, status, creation timestamp and reference metadata. Ledger rows should be append-only. Money requests should be separate from transfers because a request does **not** move funds until the payer explicitly accepts it.

A defensible request state machine is:

`PENDING → PAID | DECLINED | CANCELLED | EXPIRED`

Only `PENDING → PAID` creates a transfer, and the request must store or uniquely reference that resulting transfer so repeated acceptance cannot create multiple payments. OWASP specifically recommends enforcing permissible transaction state transitions on the server and preventing users from skipping or reordering authorization steps. [8]

For side effects such as notifications, email or activity feeds, the application should not first commit the transfer and then blindly call another service. That creates a dual-write problem: the money can commit while the notification fails, or a notification could theoretically escape for a transaction that later rolls back. AWS's transactional-outbox guidance addresses this by writing the domain change and an outbox record inside the **same database transaction**, then asynchronously publishing committed events; it also warns that downstream consumers should tolerate duplicate messages. [11]

A dedicated financial database such as TigerBeetle becomes a plausible later-stage alternative because it natively provides immutable transfers, strict financial consistency, pending/posting semantics and strict serializability. Its own architecture guidance, however, still expects a general-purpose database for user and metadata records and explicitly warns against exposing the ledger database directly to untrusted clients. [12]

For the hackathon, therefore:

| Alternative | Correctness | Build speed | Operational burden | Future scale path | Recommendation |
|---|---|---|---|---|---|
| CRUD + mutable balance | Low | Very high | Low | Weak | No |
| **PostgreSQL modular monolith + ledger** | **High** | **High** | **Low–moderate** | **Strong** | **Best choice** |

| Alternative | Correctness | Build speed | Operational burden | Future scale path | Recommendation |
|---|---|---|---|---|---|
| PostgreSQL + full event sourcing | High | Moderate | Moderate | Strong | Only if team already knows it |
| TigerBeetle + metadata DB | Very high ledger guarantees | Moderate-low | Higher | Very strong | Later-stage option |
| Microservices + distributed ledger | Potentially high | Low | Very high | Strong | Over-engineered for challenge |

The preference for the PostgreSQL option is an engineering judgment based on the competition's limited time and explicit instruction to favor a small but thoughtful working product, rather than evidence that PostgreSQL is universally superior. 🗄filecite🗄turn0file0🗄

## Risks, data gaps, and recommended next steps

The largest research limitation is the **absence of workload and service-level data**. The brief supplies a possible future population but no transfers-per-user distribution, hot-account behavior, acceptable transfer latency, availability target, recovery-point objective, recovery-time objective, daily transfer limits, fraud assumptions, geographic topology or retention policy. Consequently, the throughput and storage figures above are sensitivity scenarios rather than forecasts. 🗄filecite🗄turn0file0🗄

A second gap is that the challenge describes simulated money, so this report intentionally does not infer a real-world Bangladesh banking-license, AML/KYC, payment-rail or stored-value regulatory regime. If the same architecture were turned into a production financial product, legal and regulatory research would become a separate first-class workstream.

The highest-value next step for a hackathon team is **not adding more features; it is proving failure behavior**. The prototype should include automated tests where two requests simultaneously attempt to overspend one account, the same idempotency key is submitted repeatedly, the server "loses" its response after committing, two users transfer to one another in opposite directions, a money request is accepted twice, and the notification worker is unavailable during a successful transfer. PostgreSQL explicitly requires retry handling when serializable transactions encounter conflicts, while idempotent transaction submission is designed precisely for ambiguous retries. [13]

The team should also expose demonstrable accounting checks to judges: every completed transfer has exactly balanced debit and credit effects; no unauthorized negative balance can result from concurrency; repeated submission of the same operation never moves additional money; transfer history remains immutable; and a complete reconciliation can recompute balances from the ledger. Those properties communicate "trustworthy money software" far more convincingly than a long feature list.

Finally, load testing should be performed against measured rather than imagined usage. Begin around the illustrative ~4,630 TPS peak scenario, record p50/p95/p99 transaction latency, database lock waits,

serialization retry rate, CPU, write I/O and index growth, then increase contention by concentrating transfers on a small set of hot accounts. Only evidence from those measurements should trigger a decision to partition, introduce read replicas, separate analytical workloads, or adopt a dedicated distributed transaction engine.

The evidence assessment is therefore:

| Major finding | Confidence | Basis |
|---|---|---|
| Atomic transactions are essential for transfers | **Very high** | PostgreSQL transaction semantics; financial-ledger design; peer-reviewed transaction systems. [14] |
| Idempotency is essential under retries/timeouts | **Very high** | Financial ledger and payment API practices; prior IETF standardization work. [7] |
| Double-entry, immutable history is preferable to balance-only CRUD | **High** | Accounting-ledger guarantees and reconciliation properties. [4] |
| Authorization and transaction state transitions belong server-side | **High** | OWASP transaction-authorization guidance and ASVS security framework. [15] |
| A PostgreSQL modular monolith is the best hackathon architecture | **Medium-high** | Strong technical capability plus the challenge's restricted scope; still an implementation judgment. [1] filecite turn0file0 |
| Ten million users require immediate database sharding | **Low / unsupported** | User count alone does not specify transaction workload; measured load is missing. |
| A future high-volume system can retain strong transactional guarantees | **High conceptually; workload-specific performance unknown** | Recent peer-reviewed scalable transaction systems demonstrate feasibility, but not the exact PSTU workload. [16] |

The decisive architectural principle is therefore simple: **optimize first for invariants, not components**. A trustworthy money application is one where retries are harmless, concurrency cannot create money or overspend it, every balance has an auditable history, authorization cannot be bypassed from the client, and failure after any individual step has a defined outcome. That directly answers the challenge's invitation to think beyond CRUD while remaining achievable as a focused working product. filecite turn0file0

---

[1] [6] [13] [14] PostgreSQL: Documentation: 18: 13.2. Transaction Isolation
https://www.postgresql.org/docs/current/transaction-iso.html

[2] [4] Transfer
https://docs.tigerbeetle.com/reference/transfer/?utm_source=chatgpt.com

3 9 NIST Special Publication 800-63B
https://pages.nist.gov/800-63-4/sp800-63b.html?utm_source=chatgpt.com

5 10 Distributed Transactions at Scale in Amazon DynamoDB | USENIX
https://www.usenix.org/conference/atc23/presentation/idziorek?utm_source=chatgpt.com

7 Reliable Transaction Submission
https://docs.tigerbeetle.com/coding/reliable-transaction-submission/?utm_source=chatgpt.com

8 15 Transaction Authorization - OWASP Cheat Sheet Series
https://cheatsheetseries.owasp.org/cheatsheets/Transaction_Authorization_Cheat_Sheet.html

11 Transactional outbox pattern - AWS Prescriptive Guidance
https://docs.aws.amazon.com/en_en/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html?utm_source=chatgpt.com

12 Two-Phase Transfers
https://docs.tigerbeetle.com/coding/two-phase-transfers/?utm_source=chatgpt.com

16 ScaleDB: A Scalable, Asynchronous In-Memory Database | USENIX
https://www.usenix.org/conference/osdi23/presentation/mehdi?utm_source=chatgpt.com